lifetime. I explain service lifetimes in greater detail in chapter 7. For now, understand that services with a lifetime other than a singleton should not be injected into the constructor of a singleton. That means nonsingleton services should not be injected into the constructor of convention-based middleware. However, they can be injected into the constructor of `IMiddleware` components. Note that it is possible to inject nonsingleton services safely into the `Invoke`/`InvokeAsync` method of convention-based middleware.

I should note that the majority of framework middleware is authored using the convention-based approach. This is mainly because most of it was written before `IMiddleware` was introduced. While there is no indication that the framework designers feel any need to migrate existing components to `IMiddleware,` they recommend that you should favor `IMiddleware` for any middleware you create yourself.

We have looked in detail at how middleware is used to construct the request pipeline but haven't gone into any real detail on the middleware that has been added to the default project template. This will be covered in greater depth in forthcoming chapters. Specifically, we will look at how routing and endpoint middleware combine in chapter 4, how authorization works in chapter 10, and how to manage custom error pages in chapter 12.

Summary

- The starting point of your Razor Pages application is based on a template.
- Razor Pages applications are created as projects.
- Solutions are containers for managing projects.